

Stephen Patpatyan

CS4680 – Prompt Engineering

Dr. Amamra

Synthetic Healthcare Data Generation Agent Final Report

1. Introduction

Large language models revolutionized how software systems process and generate natural language. Early deployments used simple prompt and response patterns, where the user sends a static prompt and the model returns text. Prompt engineering emerged as a skill to create more reliable and structured outputs from these models through careful instruction design.

Agentic systems extend this capability by embedding LLMs inside control loops where they can take actions, call tools, observe results, and iterate. Rather than a single-shot response, an agentic LLM decides what to do, executes it, and adapts based on what it learns, enabling workflows that would be impossible with a static prompt alone.

Motivation

Synthetic healthcare data generation is a fundamentally structured task. Generating a valid set of ICU lab values requires precise numerical conditioning on patient demographics, a trained generative model that transforms these conditions into plausible lab distributions, inverse scaling and domain-specific post-processing, and flexible natural language interpretation so that users can describe patients in casual terms.

A static prompt cannot fulfill these requirements because a prompt alone cannot execute the model, call the scaler, or format the lab values into a clean table. It can only generate text that describes what the output might look like, which is useless for research or testing pipelines.

Problem Statement

The goal of this project is to allow a researcher or engineer to describe a patient profile in natural language and receive a set of realistic, model-generated ICU lab values in response. The system must interpret natural language patient descriptions into structured model inputs, invoke the conditional variational autoencoder (cVAE) generative model with those inputs, return formatted, clinically relevant synthetic records, and maintain conversational context across multiple requests.

Approach: Tool-Augmented LLM

This project implements a tool-augmented single-agent architecture. Claude is given a structured tool definition for the cVAE inference function. When a user's natural language request requires patient data, Claude LLM decides to call the tool, constructs the structured input, receives the generated records, and incorporates them into its own response.

This architecture was chosen over pure RAG, as no retrieval database is needed and the model itself is the knowledge source, and over multi-agent, as a single agent with one tool is sufficient for the task complexity. The tool-augmented pattern directly solves the translation problem between natural language and structured model inputs.

2. Project Overview

The system is designed to be a web-accessible conversational interface that generates synthetic ICU patient lab records on demand. The user types a natural language description of the patient profile they want. An LLM interprets the request, calls the cVAE model via a structured tool interface, and returns a formatted table of 23 first-day lab values alongside a clinical explanation.

The 23 lab values generated include core metabolic panels, blood gases, hematology, coagulation, and liver function, the complete set of labs typically measured on ICU admission, derived from the MIMIC-III database (<https://physionet.org/content/mimiciii/>).

Example Scenario

- **User:** "Give me 5 sepsis patients, female, 45 years old, SICU, long stay, with renal failure as a comorbidity."
- **Claude (internal):** Recognizes request requires generate_synthetic_patients tool. Infers: ccs_group=infectious, elix_flags={septicemia: 1, renal_failure: 1}, los_bucket=14d, age=45, gender_male=False
- **Tool Call:** POST /generate with structured JSON payload. cVAE generates 5 records from latent space conditioned on the condition vector.
- **Claude (response):** Returns table of 5 patients with 23 labs each. Explains clinical plausibility of the values given sepsis physiology.

System Classification

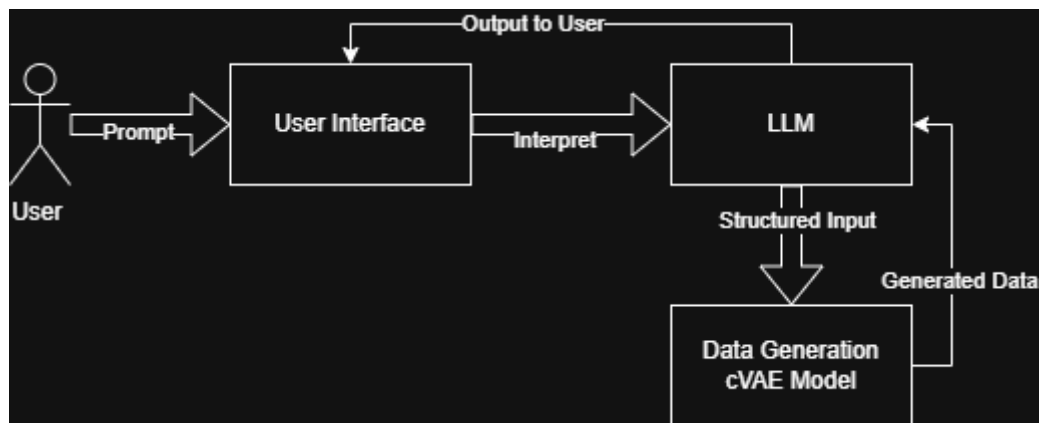
This system is a tool-augmented single-agent architecture:

- **Single Agent:** One Claude instance handles the entire conversation.
- **Tool-augmented:** Claude has access to 2 tools: `generate_synthetic_patients` and `list_valid_options`, exposed via structured JSON schemas.

3. System Architecture

The system consists of four layers:

- **Frontend (Browser):** HTML/JS chat interface. Tracks the active session ID and renders responses. Conversation history is stored server-side.
- **API Layer (FastAPI):** Receives a single new message and session ID, retrieves history from the in-memory SESSIONS storage, runs the Claude agentic loop, dispatches tool calls to the cVAE inference engine, persists the updated history, and returns the final response with the session ID.
- **LLM Layer (Claude Sonnet):** The reasoning core that reads the system prompt, interprets user messages, decides when and how to call tools, and formats the final response.
- **Model Layer (cVAE):** PyTorch generative model loaded from saved weights. Receives a conditioned latent vector and decodes it into 23 lab values.



RAG and A2A (Not Used)

This project does not use retrieval-augmented generation. RAG is appropriate when an LLM needs to ground its responses in a corpus of documents it was not trained on. In this system, the generative model is the knowledge source. There is no document corpus to retrieve from. Conditioning variables like care unit, diagnosis, comorbidities are validated against a fixed enum schema, not retrieved from a knowledge base.

A multi-agent architecture was considered but rejected for this project. The task, which is to interpret a patient description, call a model, and return results, does not require specialized agent roles. A planner/executor split would add latency and complexity without improving output quality. The single Claude instance with tool access handles interpretation, planning, execution, and formatting in a single pass.

Core Architecture

The tool-augmented pattern is the central architectural choice. Tools are exposed to Claude as JSON Schema objects within the API call. Claude receives the schema alongside the conversation and can choose to invoke a tool by emitting a structured `tool_use` block in its response.

Two tools are registered:

- `generate_synthetic_patients`: Takes age, gender, careunit, los_bucket, ccs_group, optional elix_flags, and n_samples. Returns JSON with generated patient records.
- `list_valid_options`: Returns all valid enum values for conditioning variables. Called when the user is uncertain about available options.

The key design decisions for the tool interface:

- Strict enums on categorical inputs: careunit, los_bucket, and ccs_group are enum-validated, preventing hallucinated values from reaching the model.
- Optional elix_flags as a free-form object: Allows Claude to specify any combination of the 31 Elixhauser comorbidities without requiring all 31 as explicit parameters.
- n_samples bounded 1-50: Schema-enforced limit prevents runaway generation requests.

Data Flow

The complete step-by-step pipeline for a single turn:

1. User types natural language patient description into browser chat interface.
2. Browser POSTs only the new message text and the current session ID to FastAPI /chat. Full history is never sent by the client.
3. FastAPI passes history and tool schemas to Claude API.
4. Claude reads system prompt and conversation history, determines a tool call is needed.

5. Claude emits tool_use block: {"name": "generate_synthetic_patients", "input": {...}}
6. FastAPI dispatches to build_condition_vector() -> c_scaler.transform() -> model.generate() -> x_scaler.inverse_transform() -> log-inverse for skewed labs.
7. Tool result JSON (patient records) is appended to the server-side session history as a tool_result block.
8. FastAPI persists the updated history in SESSIONS, then returns the final response text and session ID to the browser. The client immediately fetches the full history via GET /sessions/{id} to find new tool results for rendering.
9. Claude formats response text. FastAPI returns {response, messages} to browser.
10. Browser renders each tool call result as a separate labeled table. Claude's text appears below.

4. Prompt Engineering Strategy

Base Prompt

The system prompt establishes role, task, and behavioral constraints. It was designed around 3 principles: clinical grounding, tool-first behavior, and graceful assumption:

SYSTEM_PROMPT = ""You are a synthetic clinical data assistant powered by a cVAE (Conditional Variational Autoencoder) trained on MIMIC-III ICU data.

You can generate realistic synthetic first-day ICU lab records for any patient profile using the `generate\_synthetic\_patients` tool. The 23 labs cover:

- Core chemistry: glucose, sodium, potassium, creatinine, BUN, bicarbonate, chloride
- Calcium & electrolytes: calcium_total, free_calcium, phosphate
- Blood gas: pH, pCO2, pO2, lactate
- Hematology: hemoglobin, hematocrit, WBC, platelets
- Coagulation: PT, PTT
- Liver & nutrition: bilirubin, AST, albumin

When a user asks for synthetic patient data, use the tool immediately. If they are vague (e.g. "give me a sepsis patient"), make reasonable clinical assumptions and

generate the data - then explain the choices made. Always present the output in a clean, readable table format.

For queries about available options (care units, diagnoses, comorbidities), use the ``list_valid_options`` tool.

Note: All data is synthetic and for research/testing only. Never claim it represents real patients."""

Key design choices in this prompt:

- **Enumerate the 23 labs explicitly:** This prevents Claude from inventing additional labs or omitting expected ones when narrating results.
- **“Use the tool immediately”:** Without this instruction, Claude would sometimes narrate hypothetical lab values from training knowledge rather than calling the model.
- **“Make reasonable clinical assumptions”:** Enables the system to handle vague inputs (“a sepsis patient”) without forcing the user to specify all 5 required parameters.
- **Research-only disclaimer:** Explicitly scopes the use case, reducing risk of the model being used as a clinical decision aid.

Prompting for Tool Use

The most critical prompt engineering challenge was getting Claude to call the tool reliably for patient data requests, rather than generating plausible-sounding but fabricated lab values from its training data. Several iterations were required.

The instruction “use the tool immediately” was the single highest-impact addition to the system prompt. Without it, Claude would sometimes respond with statements like “A typical sepsis patient might have a lactate of 4.2 mmol/L...”, generating text that sounded like model output but was actually LLM fabrication.

The tool schema carries most of the input-formatting burden. Claude reads the enum constraints and produces valid values without additional instruction. For the `elix_flags` object, the prompt does not attempt to enumerate all 31 comorbidities. Instead, the schema description string lists them, and Claude maps natural language terms (“CHF”, “congestive heart failure”) to the correct key (“chf”).

The instruction to “explain the choices made” when inputs are vague produces responses like: “I assumed circulatory as the diagnosis group and CHF as the primary comorbidity for a cardiac profile. Here are the generated records.” This keeps the user informed and correctable without requiring them to specify every parameter upfront.

Iterative Refinement

The prompt and system evolved through several failure modes:

Failure 1: LLM hallucinating lab values

Initial problem: Claude would respond to “give me a diabetic patient” by generating plausible-sounding numbers from its training knowledge, bypassing the tool entirely.

Fix: Added explicit instruction “use the tool immediately” and framed the LLM role as interpreter/formatter rather than generator. Again, Claude is not the knowledge source for lab values. The cVAE is.

Failure 2: Invalid conditioning values

Initial problem: Claude would hallucinate careunit values like “ICU” or “general” which do not exist in the model’s training vocabulary, causing inference errors.

Fix: Moved all categoricals to strict JSON Schema enums. The LLM cannot produce an invalid careunit because the schema rejects it before the tool call reaches the model.

Failure 3: Showing only the last tool result

Initial problem: When Claude made multiple `generate_synthetic_patients` calls in one turn (like “give me 3 cardiac patients and 3 sepsis patients”), the UI only displayed the last batch.

Fix: Redesigned `extractAllPatientBatches()` to scan all new messages in the current turn, collecting every `tool_result` block and labeling each with its call parameters (careunit, ccs_group, n_samples).

Failure 4: Old tool results re-rendering on subsequent turns

Initial problem: After migrating to server-side sessions, the browser fetched the full history on every response and scanned it entirely for tool results, causing tables from prior turns to re-render alongside the current turn’s output.

Fix: Captured `history.length` (prevLen) before sending the request. After fetching the updated session history, only messages at index prevLen+1 and beyond are scanned for tool results, isolating the current turn’s batches from all prior ones.

5. Implementation

Stack

Generative Model: PyTorch cVAE (Encoder and Decoder with batch normalization)

Model training data: MIMIC-III ICU

Preprocessing: scikit-learn StandardScaler and log1p transforms for skewed labs. Very careful considerations such as recleaning data due to FLAG column filtering and excluding incomparable NICU patient data.

API server: FastAPI (Python) with CORS middleware

LLM: Claude Sonnet 4 via Anthropic API

Tool Interface: Anthropic tool_use with JSON Schema and function calling

Frontend: Standard HTML/CSS/JS

Persistence: Server-side SESSIONS storage

Architecture-Specific Notes

The model is a conditional VAE where the condition vector c encodes patient demographics and clinical context. Generation skips the encoder entirely. A random latent vector z is sampled from $N(0,1)$ and decoded alongside c :

```
def generate(self, c, n_samples=1):
```

```
    z = torch.randn(c.size(0), self.latent_dim, device=DEVICE)
```

```
    return self.decoder(z, c)
```

Post-processing applies inverse StandardScaler then $\text{np.exp}(1)$ for labs that were log-transformed during training (glucose, creatinine, lactate, PT, PTT, bilirubin, AST) to restore their original skewed distributions.

Tool Schema Definition

The tool schemas are defined as Python dicts matching the Anthropic API tool format and passed in every API call. The `elix_flags` field uses `additionalProperties` to accept any valid Elixhauser key as a 0/1 integer without requiring all 31 keys to be listed as individual parameters.

Agentic Loop

The FastAPI `/chat` endpoint runs a while loop that continues until Claude returns `stop_reason="end_turn"`. On each iteration, if Claude returns `stop_reason="tool_use"`, the server

dispatches all tool_use blocks sequentially, appends tool_result blocks to history, and calls Claude again. This enables multi-step reasoning within a single user turn.

Server-Side Session Management

Conversation history is stored entirely on the server in the SESSIONS dict, a plain Python dictionary with keys of UUID. Each entry holds the message list, an auto-generated label (first 60 chars of the opening message), and created/updated timestamps. The browser sends only the new message text and the current session ID with each request. After a response, the browser calls GET /sessions/{id} to fetch the full updated history for rendering new tool results. This gives the server authority over state, enables session recovery across browser reloads, and eliminates the payload growth problem that came with client-side history. Four REST endpoints manage sessions: GET /sessions lists all, GET /sessions/{id} restores a full conversation, DELETE /sessions/{id} removes one, and PATCH /sessions/{id}/label renames it. Users can return to a prior session by clicking it from the sidebar list or pasting a full UUID into the session ID input field.

6. Examples/Case Studies

Case Study 1: Specific Patient Profile

Input: “Generate 5 sepsis patients, female, 45 years old, SICU, long stay, with renal failure.”

Tool call constructed by Claude:

```
{
  "age":45,
  "gender_male": false,
  "careunit": "SICU",
  "los_bucket": ">14d",
  "ccs_group": "infectious",
  "elix_flags": { "septicemia": 1, "renal_failure": 1 },
  "n_samples": 5
}
```

Analysis: Claude correctly mapped “sepsis” to ccs_group=infectious and “long stay” to >14d. The elix_flags correctly included both septicemia and renal_failure. The generated

lab values showed elevated creatinine (renal failure marker), elevated lactate (sepsis marker), and reduced albumin (consistent with long ICU stay), clinically coherent outputs.

Case Study 2: Vague Input Requiring Assumption

Input: “Give me a cardiac patient.”

Claude response (before tool call): Claude assumed: male, 68 years old, CCU, circulatory diagnosis, 2-7 day stay, CHF comorbidity. Generated 5 records and stated assumptions explicitly.

Analysis: The “make reasonable clinical assumptions” instruction worked correctly. The disclosed assumptions allowed the user to immediately correct any mismatches (“actually make them younger, female”). The tool was called without prompting the user for the 5 required parameters, reducing friction significantly.

Case Study 3: Multi-Call Turn

Input: “Give me 3 cardiac patients and 3 sepsis patients so I can compare their labs.”

Claude behavior: Made two sequential tool calls, one for each profile, within the same agentic loop iteration. Each call returned 3 records.

UI rendering: Two separate labeled tables were rendered: “Call 1 - 3 patients: n=3, CCU, circulatory, age 65” and “Call 2 - 3 patients: n=3, SICU, infectious, age 45”. Claude’s response text compared the lab distributions across the two groups.

Analysis: This case revealed two bugs. First, the original `extractPatients()` function scanned history backwards and stopped at the first result, showing only the last batch. This was fixed by `extractAllPatientBatches()`, which scans all new messages in the current turn and collects every `tool_result` block into a labeled array. Second, after migrating to server-side sessions, the turn-scoping logic was updated to slice from `prevLen+1` so only the current turn’s results are rendered. The comparison use case is probably now one of the highest-value workflows the system supports.

7. Limitations

Where Prompts Fail

- **Highly specific clinical scenarios:** If a user requests a very rare combination (e.g., a patient with 8 specific comorbidities and an unusual diagnosis), Claude may map some conditions incorrectly to the nearest valid enum value without flagging the mismatch.

- **Conditioning outside training range:** The model was trained on MIMIC-III ages roughly 18-90. Requesting pediatric patients (age < 18) will produce outputs that are extrapolations outside the training distribution.
- **Diagnosis specificity:** The 19 CCS groups are high-level categories. For example, “pneumonia,” “ARDS,” and “COPD exacerbation” all map to the same respiratory category, losing diagnostic specificity that would affect lab distributions in reality.

Ambiguity Issues

- “Long stay” is interpreted as >14d, but some users mean 7-14d. The system does not clarify.
- Comorbidity language varies: “kidney disease,” “renal insufficiency,” and “renal failure” are all reasonable inputs but only the last one maps cleanly to the `elix_flags` key.

Hallucination Risks

- If Claude encounters an unfamiliar comorbidity term, it may silently omit it from `elix_flags` rather than asking for clarification, producing a less-conditioned output without disclosure.
- On very long conversations, context window pressure may cause Claude to be less precise about tool input construction as earlier conversation content is compressed.

Sensitivity to Wording

- Asking “generate 5 patients” vs “show me 5 examples” produces consistent behavior, but “what would labs look like for a sepsis patient” occasionally triggers a descriptive text response instead of a tool call, despite the instruction to use the tool immediately. Adding “always call the tool” rather than “use the tool immediately” may further reduce this.
- The number of records requested is highly sensitive: “a few patients” is interpreted as 5, but “several” is sometimes interpreted as 3 and sometimes as 7.

8. Lessons Learned

What Prompt Techniques Worked Best

- **Immediate action instructions:** “Use the tool immediately” was more effective than “you may use the tool” or “consider using the tool,” since LLMs tend to interpret optionality literally.

- **Assumption disclosure:** Instructing Claude to state its assumptions when filling in missing parameters turned a potential failure mode into a feature. Users can correct assumptions conversationally without re-specifying everything.
- **Schema-enforced constraints over prompt-enforced constraints:** Moving categoricals to JSON Schema enums was far more reliable than instructing Claude in the system prompt to “only use valid careunit values.” The schema is authoritative, while the prompt is advisory.
- **Explicit lab enumeration:** Listing all 23 labs in the system prompt prevented Claude from inventing additional labs or summarizing only a subset in its response.

What Was Surprising

- Claude’s ability to map clinical natural language (“CHF,” “congestive heart failure,” “heart failure”) to the correct `elix_flags` key without any explicit mapping table in the prompt. This reflects deep clinical knowledge in the base model.
- The multi-tool-call behavior emerged naturally without any prompt instruction. Claude decided independently to make two sequential calls when asked for two different patient profiles, and the agentic loop handled it correctly.
- The hardest bug to find was the client-side history rendering issue, not the LLM behavior. The LLM was producing correct tool calls, but the UI was discarding all but the last result.

What Would Be Done Differently

- **Add a confirmation step for large batches:** Requests for more than 20 patients should prompt Claude to confirm before generating, reducing accidental large API calls.
- **Retrain on non-credentialed data:** The MIMIC-III licensing constraint is the biggest practical limitation. Retraining on a permissively licensed EHR dataset would allow the model weights to be published and commercialized freely.
- **Add a streaming response:** The current implementation waits for the full agentic loop to be completed before returning anything. Streaming the text response while the table renders would improve the perceived responsiveness.
- **Add structured output validation:** The tool result is trusted as-is. Adding a validation step that checks generated lab values against clinical reference ranges and flags extreme outliers would increase reliability for research use.
- **Add CSV file support:** Add explicit functionality (tool call) that compiles all patients generated and create a downloadable CSV file for the user.

- **Add more persistent session storage:** This project uses a simple dict that wipes away if the server restarts. Something like Redis can be used to allow for better caching and storage.

Key Takeaways About Controlling LLM Behavior

- **Tools are better than instructions for structured tasks.** Whenever the task involves a specific format or constraint, encode it in the tool schema, not the system prompt. Schemas are parsed deterministically, while prompts are interpreted probabilistically.
- **The LLM should be the interpreter, not the generator.** In this system, the cVAE is the knowledge source for lab values. Claude's role is translation and formatting. Conflating the two roles (asking Claude to both interpret the request and generate the data) produces hallucination.
- **Conversation history is the memory system.** There is no database and no vector store. Conversation state lives in a server-side SESSIONS dict, a plain in-memory Python dictionary. This is simple and requires no external infrastructure, though history is lost if the server restarts. The client manages only a session ID string, not a growing message array.
- **Agentic loops should be built to be nicely auditable.** Exposing a GET /sessions/{id} endpoint that returns full message history allows the UI to inspect every tool call and result, render each tool batch independently, and recover any prior session by ID, all without requiring server-side logging or a persistent database.